Evolutionary Computation (Lab 5: List of Moves)

Oliwier Necelman 160300 Karol Okoń 160323

Problem story

The student still has to attend 100 of the 200 classes in one day while getting as little tired as possible. Starting from a random plan, they can either replace an attended class with one not attended, or change the route by reconnecting two edges. This experiment compares two ways of choosing these improvements: a steepest local search that scans the neighborhood each step, and a solver that keeps a sorted list of promising moves and updates it after each change.

Problem description

Given n nodes, each with coordinates and a cost, select exactly $k = \lceil n/2 \rceil$ nodes and form a Hamiltonian cycle through them. The objective is to minimize the cycle length plus the costs of the selected nodes:

$$f(C) = \sum_{i=1}^{k} d\left(C_i, C_{(i \bmod k)+1}\right) + \sum_{i=1}^{k} c(C_i),$$

where d is the distance between two nodes and c is the node cost. Distances are Euclidean distances rounded mathematically to integers. Both instances contain $n = 200$ nodes, so each solution selects $k = 100$.

Compared algorithms

Both methods start from the same type of solution: a random set of 100 nodes in random order. Both use the same two move types:

- **Inter-route move**, exchange of a selected and an unselected node: node C_i is replaced by $u \in U$. With $a = C_{i-1}$, $v = C_i$, $b = C_{i+1}$:

$$\Delta = D[a][u] + D[u][b] + c[u] - D[a][v] - D[v][b] - c[v].$$

- **Intra-route move, two-edges exchange**: edges (C_i, C_{i+1}) and (C_j, C_{j+1}) are replaced by (C_i, C_j) and (C_{i+1}, C_{j+1}), which reverses the part of the cycle between them:

$$\Delta = D[C_i][C_j] + D[C_{i+1}][C_{j+1}] - D[C_i][C_{i+1}] - D[C_j][C_{j+1}].$$

  Pairs of edges sharing a node are skipped, as the move changes nothing.

List of moves

`ListOfMovesSolver` is steepest local search that reuses the move evaluations (deltas) from previous iterations. It starts by evaluating all node-replacement and two-edges moves of the initial solution and stores the improving ones in a list of moves (LM), an ordered set with the most negative objective delta first. Each stored move describes the nodes and edges it depends on, along with its delta. In every iteration LM is browsed from the best move until an applicable one is found; that move is applied, and only the moves whose delta may have changed are evaluated again and added to LM.

The delta is the objective change, $f(C_{\text{after}}) - f(C_{\text{before}})$. Only moves with $\Delta < 0$ are stored. For a node replacement, let v be the selected node at position i, u an unselected node, and a, b the previous and next nodes. The delta is

$$\Delta_{\text{replace}} = D[a][u] + D[u][b] + c[u] - D[a][v] - D[v][b] - c[v].$$

For a two-edges move, let the current edges be (x_1, x_2) and (x_3, x_4). The proposed reconnection replaces them by (x_1, x_3) and (x_2, x_4):

$$\Delta_{\text{edges}} = D[x_1][x_3] + D[x_2][x_4] - D[x_1][x_2] - D[x_3][x_4].$$

The list is ordered by increasing delta, so its first element is the most improving move. The following pseudocode separates initialization and selection from the two move-specific procedures:

```
procedure LIST-OF-MOVES(D, c, seed):
    C <- random initial cycle using seed
    selected <- nodes in C
    L <- empty ordered set, sorted by (delta, node identifiers)

    for each position p in C:
        GENERATE-REPLACEMENTS(p)
    for each pair of non-adjacent edges in C:
        GENERATE-TWO-EDGES-MOVES(edge pair)

    repeat
        applied <- false
        for each move m in L, starting from the best:
            if m is a replacement:
                applied <- VALIDATE-AND-APPLY-REPLACEMENT(m)
            else:
                applied <- VALIDATE-AND-APPLY-TWO-EDGES(m)
            if applied:
                break
    until not applied            ▷ no applicable move in L: local optimum

    return C
```

Node-replacement moves

For each selected position, the solver tries every currently unselected node as its replacement. A stored move keeps the incoming node u, the outgoing node v, the neighbors of v, and the calculated delta. When LM is browsed, the move is applicable if v is still selected, u is still unselected, and the neighbors of v are the same pair as recorded (in either order); otherwise it can never become applicable again and is removed from LM. After the replacement, the moves are evaluated again for the changed position and its two adjacent positions (their neighbors changed), for the removed node v, which can now be inserted in place of any selected node, and for the two new edges.

```
procedure GENERATE-REPLACEMENTS(position p):
    a <- predecessor of p in C
    v <- node at p
    b <- successor of p in C
    for each node u not in selected:
        delta <- D[a][u] + D[u][b] + c[u]
                 - D[a][v] - D[v][b] - c[v]
        if delta < 0:
            insert (REPLACE, incoming=u, outgoing=v,
```

```
                    neighbors=a,b, delta) into L

procedure GENERATE-ENTERING(node u):
    for each position p in C:
        evaluate replacing the node at p by u, as above

procedure VALIDATE-AND-APPLY-REPLACEMENT(m):
    p <- current position of m.outgoing in C
    if m.outgoing is not selected or m.incoming is selected
       or {predecessor(p), successor(p)} != {m.neighbors}:
        remove m from L
        return false

    remove m from L
    replace m.outgoing at p by m.incoming
    update selected to reflect the replacement

    for each position q in {predecessor(p), p, successor(p)}:
        GENERATE-REPLACEMENTS(q)
    GENERATE-ENTERING(m.outgoing)
    GENERATE-TWO-EDGES-MOVES(edge predecessor(p)->p)
    GENERATE-TWO-EDGES-MOVES(edge p->successor(p))
    return true
```

Generating moves for an edge means pairing it with each other non-adjacent cycle edge and testing both possible reconnections. The predecessor and successor are interpreted in the updated cycle.

Two-edges moves

The initial generation considers all pairs of non-adjacent cycle edges. For each pair it evaluates both ways to reconnect the four endpoints: the currently applicable one and the one with relatively inverted edges, which would split the cycle now but may become applicable after a later reversal. When LM is browsed, a stored two-edges move is in one of three situations:

1. At least one of the removed edges no longer exists in the current solution: the move is removed from LM.
2. Both edges exist, but in a different relative direction than the saved one (exactly one of them is reversed): the move is not applicable now, but it may become applicable in the future, so it stays in LM and browsing continues.
3. Both edges exist in the same relative direction (also both reversed): the move is applied and removed from LM.

After a two-edges move, replacements are evaluated at the four affected boundary positions, and new two-edges moves (both reconnections) are evaluated for the two new edges. The edges inside the reversed segment are only reversed, not created anew, so their moves are not evaluated again.

```
procedure GENERATE-TWO-EDGES-MOVES(edge (x1,x2)):
    for each cycle edge (x3,x4) non-adjacent to (x1,x2):
        for each added-edge pair in
            { ((x1,x3),(x2,x4)), ((x1,x4),(x2,x3)) }:
            delta <- sum(lengths of added edges)
                     - D[x1][x2] - D[x3][x4]
            if delta < 0:
                insert (TWO-EDGES, endpoints=x1,x2,x3,x4,
                        added-edge pair, delta) into L

procedure VALIDATE-AND-APPLY-TWO-EDGES(m):
    if (m.x1,m.x2) or (m.x3,m.x4) is not an edge of C:     ▷ situation 1
        remove m from L
```

```
    return false
if the two edges have different relative directions:   ▷ situation 2
    keep m in L
    return false
remove m from L                                        ▷ situation 3
reverse the cycle segment between the two edges
for each of the four affected boundary positions:
    GENERATE-REPLACEMENTS(position)
GENERATE-TWO-EDGES-MOVES(first new boundary edge)
GENERATE-TWO-EDGES-MOVES(second new boundary edge)
return true
```

After each applied move, all moves whose delta may have changed are evaluated again: moves removing a new edge, replacements of nodes with new neighbors, and moves involving a node that entered or left the cycle. All other moves keep their deltas, which are taken from LM. The search ends when LM contains no applicable move; at that point no improving move exists in the whole neighborhood, so the result is a local optimum of the same neighborhood as in the baseline. We use this incremental variant instead of the advised one that re-evaluates all moves only when LM contains no applicable move.

Computational experiment

Each method was run 200 times per instance; the methods with a random start use seeds 1–200, so the baseline and the list of moves start from the same 200 random solutions. All earlier methods were run again in the same experiment. The objective values, running times, and step counts below are taken from `results/lab5_summary.csv`. A step is one improving move applied. Running time is the average time per `solve` call in milliseconds and includes creation of the random starting solution. The experiment runner compiles with `g++` and `-O2`.

Lower objective values are better. The tables show the average and the minimum–maximum range over the runs.

Objective function

Method	Instance A	Instance B
	av (min – max)	av (min – max)
Random solution	260 176 (229 522 – 286 183)	205 847 (186 703 – 227 217)
Nearest neighbor I	87 477 (84 362 – 90 075)	59 397 (56 007 – 63 166)
Nearest neighbor II	74 307 (72 328 – 77 022)	59 476 (52 915 – 66 164)
Greedy cycle	73 739 (72 590 – 75 281)	59 651 (55 655 – 63 734)
Nearest neighbor II 2-regret	122 861 (111 527 – 133 181)	74 763 (69 032 – 80 767)
Nearest neighbor II weighted 2-regret	75 561 (72 419 – 78 504)	55 698 (50 485 – 61 406)
Greedy cycle 2-regret	120 995 (112 654 – 130 391)	73 459 (69 609 – 78 175)
Greedy cycle weighted 2-regret	73 353 (72 011 – 75 532)	57 863 (52 731 – 63 370)
LS greedy, two-nodes, random start	88 483 (82 255 – 95 221)	64 546 (58 800 – 71 074)
LS greedy, two-nodes, greedy start	73 020 (71 955 – 74 877)	57 523 (52 702 – 62 933)
LS greedy, two-edges, random start	75 508 (73 219 – 78 672)	52 356 (49 714 – 54 905)
LS greedy, two-edges, greedy start	72 960 (71 955 – 74 877)	57 198 (52 542 – 62 572)
LS steepest, two-nodes, random start	90 263 (83 393 – 100 272)	66 253 (58 629 – 72 897)
LS steepest, two-nodes, greedy start	73 022 (71 955 – 74 861)	57 525 (52 702 – 62 933)
LS steepest, two-edges, random start	76 228 (73 629 – 81 989)	52 606 (50 280 – 55 256)
LS steepest, two-edges, greedy start	73 004 (71 955 – 74 861)	57 126 (52 542 – 62 301)
LS with candidates	76 279 (74 031 – 81 012)	52 720 (49 892 – 55 894)
LS with list of moves	76 237 (73 629 – 81 989)	52 603 (50 280 – 55 256)

Running time [ms]

Method	Instance A	Instance B
	av (min – max)	av (min – max)
Random solution	0.010 (0.008 – 0.020)	0.009 (0.008 – 0.019)
Nearest neighbor I	0.092 (0.073 – 0.200)	0.090 (0.071 – 0.248)
Nearest neighbor II	2.876 (2.312 – 4.366)	2.817 (2.308 – 3.516)
Greedy cycle	2.338 (2.004 – 2.777)	2.328 (1.902 – 2.848)
Nearest neighbor II 2-regret	2.732 (2.143 – 3.493)	2.787 (2.049 – 3.893)
Nearest neighbor II weighted 2-regret	2.879 (2.204 – 3.862)	2.980 (2.181 – 4.197)
Greedy cycle 2-regret	3.827 (3.133 – 4.966)	4.207 (3.107 – 6.398)
Greedy cycle weighted 2-regret	3.805 (3.055 – 5.011)	3.832 (2.944 – 5.575)
LS greedy, two-nodes, random start	8.189 (4.237 – 15.288)	5.728 (3.051 – 10.748)
LS greedy, two-nodes, greedy start	4.413 (3.310 – 7.532)	4.483 (3.417 – 6.026)
LS greedy, two-edges, random start	7.055 (3.475 – 10.774)	4.496 (2.369 – 7.854)
LS greedy, two-edges, greedy start	4.295 (3.470 – 6.760)	4.611 (3.290 – 6.479)
LS steepest, two-nodes, random start	21.217 (15.778 – 29.577)	21.392 (16.111 – 29.804)
LS steepest, two-nodes, greedy start	4.349 (3.314 – 5.914)	4.918 (3.578 – 7.145)
LS steepest, two-edges, random start	13.494 (10.486 – 19.403)	14.263 (11.303 – 21.847)
LS steepest, two-edges, greedy start	4.316 (2.866 – 6.170)	4.847 (3.453 – 7.542)
LS with candidates	8.515 (7.143 – 11.844)	7.793 (6.365 – 10.295)
LS with list of moves	8.117 (6.316 – 17.589)	7.558 (6.021 – 9.417)

Number of steps

Method	Instance A	Instance B
	av (min – max)	av (min – max)
LS greedy, two-nodes, random start	523.2 (447 – 614)	520.2 (413 – 622)
LS greedy, two-nodes, greedy start	4.3 (0 – 14)	5.9 (1 – 34)
LS greedy, two-edges, random start	508.2 (439 – 582)	514.2 (429 – 566)
LS greedy, two-edges, greedy start	5.2 (0 – 23)	10.2 (2 – 34)
LS steepest, two-nodes, random start	152.3 (117 – 186)	150.2 (125 – 194)
LS steepest, two-nodes, greedy start	3.3 (0 – 9)	5.3 (1 – 21)
LS steepest, two-edges, random start	132.9 (115 – 149)	133.2 (119 – 158)
LS steepest, two-edges, greedy start	3.3 (0 – 10)	9.2 (2 – 24)
LS with candidates	132.2 (117 – 150)	132.8 (114 – 154)
LS with list of moves	132.8 (115 – 149)	133.2 (117 – 158)

Visualization of the best solutions

Each panel shows the best solution of the baseline (left) and of local search with the list of moves (right); they are the same cycle. Node color encodes the node cost (darker means more expensive). Selected nodes are drawn larger, the solid line shows the cycle, and the dotted line marks the edge from the last node back to the first.

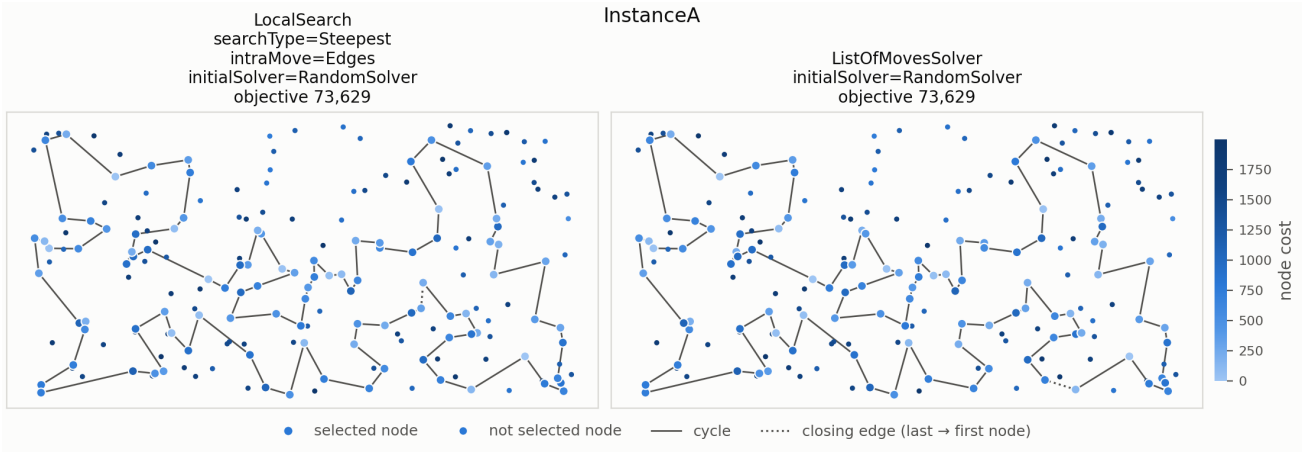


Figure 1: Best solutions for instance A

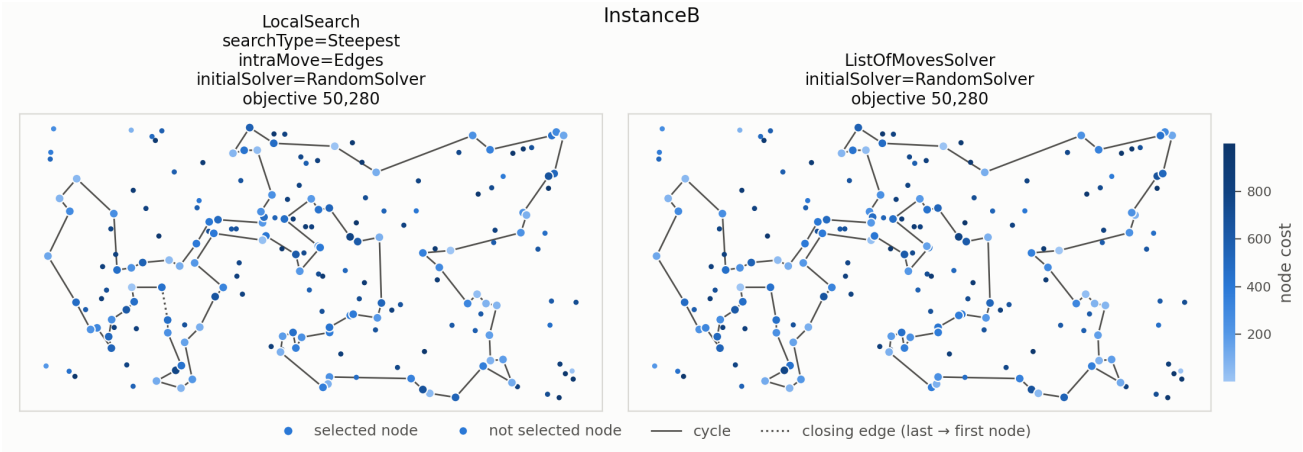


Figure 2: Best solutions for instance B

Best solutions

The best solutions of the earlier methods are listed in the other lab reports. On both instances, the best solution of the list of moves is the same cycle as the best solution of the baseline (from the same starting solution).

Instance A

LS steepest, two-edges, random start and LS with list of moves (objective 73 629)

```
174 150 106  89  42 151 138 182  57 107  88  78 193 153 170 159 130  47  93  50
  3 169   1 178 136 186 164 142   8  54   9  33 131  99  36 177  52 179  53 140
 73  45  87 102 185 144  67  16  51  35 156  28 180  90  83 160 123 100 132  76
 71 157  91  80  48  31  60 108 152  49  63 149 116 168  95  97 127  69 129 104
 92 171   2 161  30 187 189 141  98  25 114 135 181 192 158  58  84 134  72 176
```

Instance B

LS steepest, two-edges, random start and LS with list of moves (objective 50 280)

```
 97 170 115 180   6 197  36  29 132 178  65 145  28 149 158 177 125  26 154 140
 38  30 183 172 136 133 120  96 198 121  58  48  89  57 190  70  81  94 105  61
```

```
139 196 114  10  54 162 192  80 127 129  90 153  20  14   2  74  17 171  39  99
179  87  82  85  59  73  51 167 181 152  34  24  67  66  76 135  75  86  46  72
117 184   3 151 122 160  69  21  47  64  33  40  60  45 134 175 142 157  71 155
```

Solution checker

The best solutions have been checked with the Solution checker.xlsx provided on ekursy, and we confirmed that the values computed by the checker match those computed by our algorithms.

Conclusions

- **The list of moves gives the same quality as the baseline.** The averages are practically equal (76 237 vs 76 228 on instance A, 52 603 vs 52 606 on instance B), and the best and the worst solutions are identical. In 183 (A) and 168 (B) of the 200 runs both methods end with the same solution (same objective and the same selected nodes); in the other runs they choose a different move among moves with an equal delta at some step, which leads to a different local optimum.
- **The list of moves is faster.** It takes 8.12 ms instead of 13.49 ms on instance A (1.7 times faster) and 7.56 ms instead of 14.26 ms on instance B (1.9 times faster).
- **Both methods apply the same number of moves** (132.8 vs 132.9 on instance A, 133.2 for both on instance B). The gain comes only from reusing the deltas: instead of evaluating the whole neighborhood in every step, only the moves around the changed part of the cycle are evaluated again.
- **Keeping moves with relatively inverted edges is necessary.** An earlier version of our implementation deleted these moves (and did not evaluate the moves bringing back a removed node). It stopped before reaching a local optimum in 92 (A) and 117 (B) of the 200 runs, and its results were worse than those of the baseline.
- **Compared with candidate moves, the list of moves is about as fast but does not lose quality.** Local search with candidate moves takes 8.52 ms (A) and 7.79 ms (B), but its average is slightly worse than the baseline (76 279 and 52 720), because it skips some improving moves.

Source code and data

The list-of-moves implementation is in `src/lab5_algos/ListOfMovesSolver.cpp`. The local-search implementation is in `src/lab3_algos/`. Experiment results are in `results/lab5_summary.csv` and `results/lab5_best.csv`; they are produced by `python3 scripts/run_experiment.py --name lab5`.